

ASTRA AI Desktop Agent - Technology Roadmap

1. Introduction

ASTRA is an AI desktop agent built using Node.js, React, and a Python-based AI engine. It combines system automation, real-time communication, and LLM intelligence.

2. Core Architecture Overview

ASTRA is divided into 4 major parts:

- UI Layer (Electron + React)
- Backend Layer (Node.js)
- AI Brain (Python microservice)
- System Automation Layer (Node + OS tools)

3. Step 1: Frontend (Electron + React)

Learn React for building UI components such as chat interface, command panel, and system dashboard.

Electron is used to convert React web app into a desktop application.

Key concepts: components, hooks, state management, IPC communication.

4. Step 2: Backend (Node.js)

Node.js acts as the central orchestrator of ASTRA.

Use Fastify or Express to handle APIs and WebSocket connections.

Key responsibilities:

- Handle user commands
- Manage WebSocket communication
- Execute system-level operations

5. Step 3: Real-Time Communication

Use WebSockets (socket.io or ws) for real-time interaction between UI and backend.

This enables live chat, command execution, and system logs streaming.

6. Step 4: AI Brain (Python Service)

Python handles all AI-related logic including LLM API calls and reasoning engine.

Keep AI separate from Node.js for modular architecture.

Use FastAPI for communication between Node and Python.

7. Step 5: Tool System (Core of ASTRA)

ASTRA uses a tool-based system where AI decides actions like opening apps or running commands.

Each tool is a structured function executed by Node.js backend.

Example tools: open_app, search_file, run_command, system_status

8. Step 6: System Automation

Node.js uses child_process and system libraries to control OS.

Optional Python automation with pyautogui for advanced control.

9. Step 7: Memory System

Start with SQLite or MongoDB for storing user data and command history.

Later upgrade to vector databases like ChromaDB for semantic memory.

10. Final Architecture

UI (Electron + React) → Node.js Backend → Python AI Engine → System Tools + Memory

11. Learning Path Summary

Step 1: React + Electron UI basics

Step 2: Node.js backend + APIs

Step 3: WebSockets communication

Step 4: Python AI service integration

Step 5: Tool system design

Step 6: System automation

Step 7: Memory + scaling system